CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA PAULA SOUZA ETEC PROFª ILZA NASCIMENTO PINTUS

Técnico em Desenvolvimento de Sistemas

Samuel Domingos da Silva Melo
Pietro Oliveira Silva
Pedro Vieira de Paula
Wenderson Candido Sousa Leal

TRADUTOR DE LIBRAS: sistema web de reconhecimento de sinais em tempo real utilizando visão computacional

São José dos Campos - SP
2026

Samuel Domingos da Silva Melo
Pietro Oliveira Silva
Pedro Vieira de Paula
Wenderson Candido Sousa Leal

TRADUTOR DE LIBRAS: sistema web de reconhecimento de sinais em tempo real utilizando visão computacional

Trabalho de Conclusão de Curso apresentado ao Curso Técnico em Desenvolvimento de Sistemas da Etec Profª Ilza Nascimento Pintus de São José dos Campos, orientado pelo Prof. Jean Carlos Lourenço Costa, como requisito parcial para obtenção do título de Técnico em Desenvolvimento de Sistemas.

São José dos Campos - SP
2026

Dedicamos este trabalho às pessoas surdas, que nos inspiraram a buscar, por meio da tecnologia, novas formas de aproximar mundos e derrubar barreiras de comunicação.

AGRADECIMENTOS

Agradecemos ao professor orientador Jean Carlos pela dedicação e pelas orientações essenciais ao longo do desenvolvimento deste trabalho. Agradecemos também à Etec Profª Ilza Nascimento Pintus, por toda a estrutura e o suporte oferecidos durante o curso Técnico em Desenvolvimento de Sistemas, além de familiares e amigos que apoiaram a nossa trajetória.

“A tecnologia é mais poderosa quando aproxima as pessoas.”

(Autoria própria)

RESUMO

Este trabalho apresenta o desenvolvimento do *Tradutor de Libras* (Librator), um sistema web que reconhece, em tempo real e por meio de uma *webcam* comum, letras e números da Língua Brasileira de Sinais (Libras) e os converte em texto, além de traduzir texto em Libras por meio do avatar do VLibras. A proposta busca reduzir a barreira de comunicação entre pessoas surdas e ouvintes, sem depender de sensores ou equipamentos especializados. O sistema foi desenvolvido em Python, com OpenCV para a captura de vídeo, MediaPipe para a detecção da mão e a extração de 21 pontos de referência, Scikit-learn para a classificação com o algoritmo *Random Forest* e Flask no *back-end*. O *dataset* reúne 8.839 amostras das 26 letras do alfabeto e dos números de 0 a 9, e a interface permite montar palavras por meio de gestos de espaço e de exclusão de caractere, com filtro de confiança, votação entre as predições e tempo de permanência estável para reduzir erros. Nos testes, o sistema atendeu integralmente a 11 dos 12 requisitos definidos e parcialmente ao restante, com 100% de aprovação nos testes funcionais. O classificador obteve 99% de acurácia no conjunto de teste, formado por 1.768 amostras, e 98,57% na validação cruzada, e o processamento de um quadro levou, em média, 73,9 ms, o que corresponde a cerca de 13 quadros por segundo. Um avaliador externo ao grupo aprovou todos os critérios avaliados. Entre as limitações, destacam-se o reconhecimento restrito a sinais estáticos, o alcance de uso de cerca de 1 metro, a dependência da câmera e a avaliação feita por uma única pessoa externa, sem a participação de pessoas surdas. Conclui-se que a combinação entre o MediaPipe e o *Random Forest* é adequada para o reconhecimento de sinais estáticos da Libras em tempo real.

Palavras-chave: Libras. Visão computacional. Aprendizado de máquina. *Random Forest*. MediaPipe.

ABSTRACT

This work presents the development of *Tradutor de Libras* (Librator), a web system that recognizes, in real time and through a common webcam, letters and numbers of the Brazilian Sign Language (Libras) and converts them into text, in addition to translating text into Libras through the VLibras avatar. The proposal aims to reduce the communication barrier between deaf and hearing people without relying on sensors or specialized equipment. The system was developed in Python, using OpenCV for video capture, MediaPipe for hand detection and the extraction of 21 landmarks, Scikit-learn for classification with the Random Forest algorithm, and Flask on the back end. The dataset gathers 8,839 samples of the 26 letters of the alphabet and the numbers from 0 to 9, and the interface allows users to build words through space and delete gestures, with a confidence threshold, voting among predictions and a stable-hold time to reduce errors. In the tests, the system fully met 11 of the 12 defined requirements and partially met the remaining one, with 100% approval in the functional tests. The classifier achieved 99% accuracy on the test set, made up of 1,768 samples, and 98.57% in cross-validation, and processing a frame took 73.9 ms on average, which corresponds to about 13 frames per second. An evaluator external to the group approved all the evaluated criteria. Among the limitations are the restriction to static signs, a usable range of about 1 meter, the dependence on the camera, and the evaluation by a single external person, without the participation of deaf people. It is concluded that the combination of MediaPipe and Random Forest is suitable for real-time recognition of static Libras signs.

Keywords: Libras. Computer vision. Machine learning. Random Forest. MediaPipe.

LISTA DE ABREVIATURAS E SIGLAS

API — *Application Programming Interface*

CSV — *Comma-Separated Values*

FPS — *Frames Per Second*

IA — Inteligência Artificial

IDE — *Integrated Development Environment*

JSON — *JavaScript Object Notation*

LGPD — Lei Geral de Proteção de Dados Pessoais

RF — Requisito Funcional

RNF — Requisito Não Funcional

UX/UI — *User Experience / User Interface*

LISTA DE QUADROS

Quadro 1 – Comparação técnica entre soluções de tradução de Libras.....17
Quadro 2 – Situação dos requisitos do sistema.....30
Quadro 3 – Casos de teste funcionais.....32
Quadro 4 – Métricas de desempenho do classificador e do sistema 32
Quadro 5 – Problema identificado durante o desenvolvimento e correção aplicada..35
Quadro 6 – Retorno do avaliador externo que testou o sistema 36
Quadro 7 – Fontes consultadas e onde foram utilizadas no trabalho 41

SUMÁRIO

1 INTRODUÇÃO 12
1.1 Contextualização 12
1.2 Justificativa e relevância do estudo 13
1.3 Problema e lacuna identificada 14
1.4 Questão de pesquisa 14
1.5 Objetivo geral e objetivos específicos 14
1.6 Delimitação do tema 15
1.7 Metodologia (visão geral) 15
1.8 Estrutura do trabalho 16
2 FUNDAMENTAÇÃO TEÓRICA 16
2.1 Língua Brasileira de Sinais (Libras) 16
2.2 Visão computacional 19
2.3 Detecção de mãos e extração de pontos de referência (*landmarks*) 19
2.4 Aprendizado de máquina e classificação supervisionada 20
2.5 O classificador Random Forest e a biblioteca Scikit-learn 20
2.6 Trabalhos correlatos 21
2.7 Arquitetura de sistemas web 23
2.8 Segurança e privacidade de dados em sistemas de visão computacional 23
3 DESENVOLVIMENTO 25
3.1 Requisitos do sistema 25
3.2 Ambiente e ferramentas de desenvolvimento 26
3.3 Arquitetura do sistema 27
3.4 Construção do *dataset* 27
3.5 Pré-processamento e treinamento do classificador 28
3.6 Implementação da interface web e montagem de palavras 29
3.7 Integração com o VLibras para tradução de texto em Libras 31
3.8 Considerações de segurança na implementação 31
4 RESULTADOS 33
4.1 Funcionalidades implementadas 33
4.2 Resultados dos testes 35
4.3 Avaliação do sistema por avaliador externo 38
4.4 Discussão 39
5 CONCLUSÕES 42
5.1 Síntese dos resultados 42

5.2 Limitações 43
5.3 Contribuições e trabalhos futuros 43
REFERÊNCIAS 44

1 INTRODUÇÃO

Comunicar-se é uma necessidade humana fundamental, e a linguagem constitui o principal instrumento por meio do qual o indivíduo se insere socialmente e constrói sua identidade (QUADROS; KARNOPP, 2004). Para grande parte das pessoas surdas, no entanto, essa tarefa do dia a dia esbarra em uma barreira concreta: a maioria das pessoas ouvintes desconhece a Língua Brasileira de Sinais (Libras), língua oficialmente reconhecida no país, mas ainda pouco difundida entre a população em geral (BRASIL, 2002). Essa distância se reflete em situações cotidianas, como atendimentos de saúde, ambientes escolares e relações de trabalho, e contribui para o isolamento social e para dificuldades no acesso a serviços básicos (BRASIL, 2005).

Segundo o Censo Demográfico realizado pelo Instituto Brasileiro de Geografia e Estatística, cerca de 9,7 milhões de brasileiros declaram possuir algum grau de perda auditiva (IBGE, 2010) e, em escala global, a Organização Mundial da Saúde estima que mais de 1,5 bilhão de pessoas convivam atualmente com alguma forma de perda auditiva (OMS, 2021) — números que evidenciam a dimensão do problema de comunicação enfrentado por essa parcela da população. É nesse cenário que a tecnologia, sobretudo a visão computacional e a inteligência artificial, ganha relevância: estudos recentes demonstram que a combinação entre extração de pontos de referência da mão e classificadores de aprendizado de máquina é uma abordagem viável para a tradução automática de sinais em texto, aproximando pessoas surdas e ouvintes (SINCAN; KELES, 2020; SELVARAJ et al., 2022). Este trabalho apresenta o desenvolvimento de um tradutor de Libras, um sistema web que reconhece padrões da mão captados por *webcam* para identificar letras do alfabeto em Libras e convertê-las em texto, em tempo real.

1.1 Contextualização

De acordo com o Censo Demográfico do Instituto Brasileiro de Geografia e Estatística, existem cerca de 9,7 milhões de pessoas com algum grau de perda auditiva no Brasil (IBGE, 2010), e uma parcela significativa desse grupo depende da Libras como principal ou única forma de comunicação. Segundo Quadros e Karnopp

(2004), a Libras possui estrutura gramatical própria, independente da Língua Portuguesa, o que reforça a necessidade de políticas públicas e de tecnologias específicas de apoio à comunidade surda.

A língua é reconhecida oficialmente no Brasil desde a Lei nº 10.436, de 24 de abril de 2002 (BRASIL, 2002), mas, como observa Strobel (2008), essa oficialização não se traduziu, na prática, em maior familiaridade da população ouvinte com a Libras, o que ainda gera barreiras recorrentes em atendimentos, escolas e ambientes de trabalho. Em âmbito internacional, a Organização Mundial da Saúde estima que mais de 1,5 bilhão de pessoas convivam atualmente com algum grau de perda auditiva (OMS, 2021), reforçando que a barreira de comunicação enfrentada pela comunidade surda não é uma particularidade brasileira, mas um desafio de alcance mundial.

Ao mesmo tempo, a evolução da visão computacional e do aprendizado de máquina tornou possível, mesmo em projetos de menor escala, reconhecer gestos das mãos em tempo real a partir de uma *webcam* comum, sem depender de sensores ou equipamentos especializados (ZHANG et al., 2020). É nesse cenário que este projeto se insere: aplicar, de forma prática, os conceitos de programação, visão computacional e aprendizado de máquina trabalhados no curso Técnico em Desenvolvimento de Sistemas para propor uma solução concreta de apoio à comunicação em Libras.

1.2 Justificativa e relevância do estudo

A justificativa para o desenvolvimento deste sistema passa por três frentes. No campo social, o projeto contribui para reduzir a barreira de comunicação entre pessoas surdas e ouvintes, favorecendo a inclusão em situações cotidianas e combatendo os processos de exclusão historicamente enfrentados por essa comunidade (STROBEL, 2008). No campo acadêmico, ele permite colocar em prática, em um caso real, conceitos de programação orientada a objetos, processamento de imagens, extração de características e aprendizado de máquina supervisionado, estudados ao longo do curso Técnico em Desenvolvimento de Sistemas. Já do ponto de vista prático, o uso de bibliotecas gratuitas e de uma *webcam* comum torna a solução acessível, sem exigir *hardware* especializado nem sensores de alto custo, ampliando seu potencial de uso por escolas, pequenas empresas e pela própria comunidade surda.

1.3 Problema e lacuna identificada

O mercado já conta com aplicativos e ferramentas de tradução de Libras, como o Hand Talk e o VLibras, alguns deles bastante completos. Grande parte dessas soluções, no entanto, depende de aplicativos móveis fechados, de conexão constante com serviços em nuvem ou de bases de dados extensas e proprietárias. A lacuna identificada, portanto, não está na ausência de tradutores de Libras, mas na falta de uma solução simples e de concepção aberta: executável a partir de uma *webcam* comum, construída com bibliotecas gratuitas de visão computacional e aprendizado de máquina, e capaz de servir também como material de estudo e como base para expansões futuras por outros desenvolvedores e pesquisadores.

1.4 Questão de pesquisa

É possível desenvolver um sistema web capaz de reconhecer, em tempo real, letras do alfabeto e números em Libras a partir de imagens capturadas por *webcam*, convertendo-os em texto com precisão suficiente para apoiar a comunicação entre pessoas surdas e ouvintes?

1.5 Objetivo geral e objetivos específicos

O objetivo geral deste trabalho é desenvolver um sistema web de tradução de Libras, capaz de reconhecer, em tempo real, letras do alfabeto e números em Libras capturados por *webcam* e convertê-los automaticamente em texto, além de oferecer a tradução no sentido inverso, de texto para Libras, por meio da integração com o avatar tridimensional do VLibras. Para alcançá-lo, foram definidos os seguintes objetivos específicos:

a) Estudar as técnicas de visão computacional e detecção de mãos aplicáveis ao
reconhecimento de sinais;

b) Construir um *dataset* próprio de sinais, extraindo os 21 pontos de referência da mão
para cada letra do alfabeto e para cada número;

c) Treinar um modelo de aprendizado de máquina (*RandomForestClassifier*) capaz de classificar
corretamente os sinais capturados;

d) Implementar a montagem automática de palavras, incluindo os gestos de espaço e de

exclusão de caractere;

e) Integrar o sistema a uma interface web que exiba o vídeo da *webcam* e o texto

traduzido em tempo real, além de oferecer a tradução de texto para Libras por meio do VLibras;

f) Realizar testes funcionais das principais rotinas do sistema, validando sua precisão e

usabilidade.

1.6 Delimitação do tema

O escopo deste trabalho, no sentido Libras → texto, está limitado ao reconhecimento de letras do alfabeto da Libras e de números, identificados a partir da configuração da mão em cada quadro e de sua variação em relação ao quadro anterior, sem a modelagem de sequências longas de movimento. Não fazem parte desta versão o reconhecimento de sinais dinâmicos (palavras representadas por movimento), a tradução de frases completas com a gramática própria da Libras e o reconhecimento simultâneo com as duas mãos. No sentido inverso, de texto para Libras, o sistema não implementa um tradutor próprio, e sim integra o avatar tridimensional do VLibras, solução pública desenvolvida por terceiros. O sistema também não trata do reconhecimento de expressões faciais, que na Libras também carregam valor gramatical. Essas limitações são retomadas no Capítulo 5 como sugestões para trabalhos futuros.

1.7 Metodologia (visão geral)

Este trabalho caracteriza-se como uma pesquisa aplicada, de natureza qualitativa e exploratória, conduzida em quatro etapas principais: levantamento bibliográfico sobre visão computacional, detecção de mãos e aprendizado de máquina; construção de um *dataset* próprio de sinais da Libras; implementação do sistema de captura, extração de características, classificação, montagem de frases e integração com o VLibras; e, por fim, testes funcionais para avaliar a precisão e o comportamento do sistema em tempo real. O detalhamento técnico completo, com as

ferramentas, bibliotecas e ambiente de desenvolvimento utilizados, é apresentado no Capítulo 3.

1.8 Estrutura do trabalho

Além desta introdução, o trabalho está organizado em mais quatro capítulos. O Capítulo 2 traz a fundamentação teórica que sustenta as escolhas técnicas do projeto, abordando os conceitos de visão computacional, detecção de mãos, aprendizado de máquina, arquitetura de sistemas web e segurança de dados utilizados no desenvolvimento do sistema. O Capítulo 3 apresenta o desenvolvimento propriamente dito: os requisitos, a arquitetura e a implementação do tradutor de Libras. O Capítulo 4 expõe os resultados obtidos nos testes funcionais e discute o desempenho do sistema. Por fim, o Capítulo 5 traz as conclusões do trabalho e as sugestões para pesquisas futuras.

2 FUNDAMENTAÇÃO TEÓRICA

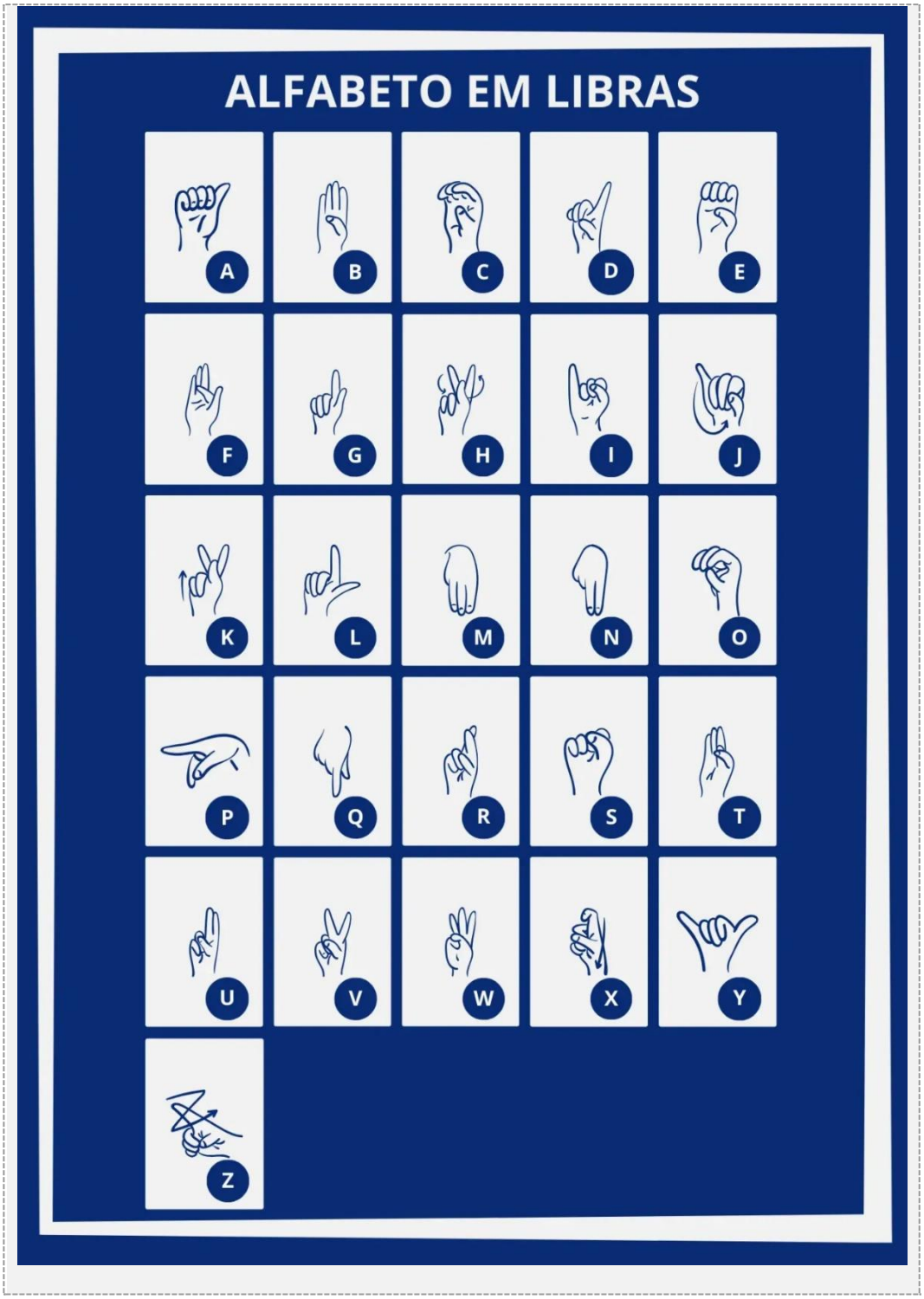
Este capítulo apresenta os conceitos teóricos que fundamentam as escolhas técnicas realizadas no desenvolvimento do tradutor de Libras. São abordados, na sequência, a Língua Brasileira de Sinais e seu reconhecimento legal, os fundamentos da visão computacional, as técnicas de detecção de mãos e extração de pontos de referência, os conceitos de aprendizado de máquina e de classificação supervisionada, o funcionamento do classificador *Random Forest* utilizado neste projeto, os trabalhos correlatos identificados na literatura e no mercado, a arquitetura típica de sistemas web e, por fim, aspectos de segurança e privacidade relevantes para sistemas que processam imagens capturadas por *webcam*.

2.1 Língua Brasileira de Sinais (Libras)

A Língua Brasileira de Sinais (Libras) é uma língua natural, de modalidade visual-motora, dotada de estrutura gramatical própria — com níveis fonológico, morfológico, sintático e semântico independentes da Língua Portuguesa —, utilizada principalmente pela comunidade surda brasileira como forma de comunicação (QUADROS; KARNOPP, 2004). No Brasil, a Libras foi reconhecida oficialmente como meio legal de comunicação e expressão pela Lei nº 10.436, de 24 de abril de 2002 (BRASIL, 2002), e teve sua regulamentação detalhada pelo Decreto nº 5.626, de 22

de dezembro de 2005 (BRASIL, 2005), que estabeleceu diretrizes para o ensino da língua, a formação de intérpretes e a garantia de acessibilidade em serviços públicos e privados. Entre os recursos que compõem a Libras está o alfabeto manual, também chamado de datilologia, utilizado para soletrar nomes próprios, termos técnicos ou palavras que ainda não possuem um sinal específico consolidado (QUADROS; KARNOPP, 2004). Diferentemente da maior parte dos sinais da língua, que costumam envolver movimento, configuração de mão, ponto de articulação e expressões não manuais simultaneamente, as letras do alfabeto manual são, em sua maioria, representadas por configurações estáticas da mão, o que as torna tecnicamente mais viáveis de serem reconhecidas por sistemas de visão computacional a partir de uma única captura de imagem. Algumas letras, como H, J, K, X e Z, envolvem movimento; para reconhecê-las, o vetor de características inclui, além da posição dos pontos da mão, a variação desses pontos em relação ao quadro anterior, conforme descrito na Seção 3.4. Por esse motivo, o trabalho delimita seu escopo ao reconhecimento do alfabeto manual e dos números, conforme já apresentado na Seção 1.6. O alfabeto manual completo é ilustrado na Figura 1, a seguir.

Figura 1 - Alfabeto manual da Libras



Fonte: Pro Atitude Educacional (2025).

A Figura 1 apresenta as configurações de mão que representam as letras do alfabeto manual da Libras, cada uma acompanhada da letra correspondente. Observa-se que a maior parte das letras é formada por posições estáticas dos dedos e da palma, o que permite identificá-las a partir de uma única imagem, sem depender de movimento. Algumas letras, no entanto, possuem configurações bastante

semelhantes entre si, o que pode gerar confusões no reconhecimento automático. Por isso, a figura serve como referência visual para a construção do *dataset* e para a avaliação do desempenho do sistema, descritas nos capítulos seguintes.

2.2 Visão computacional

A visão computacional é a área da ciência da computação dedicada a permitir que sistemas computacionais extraiam informações relevantes a partir de imagens ou vídeos, reproduzindo artificialmente capacidades de interpretação visual próprias do ser humano (GONZALEZ; WOODS, 2010). De forma geral, um sistema de visão computacional é estruturado em etapas sequenciais: aquisição da imagem, pré-processamento, segmentação, extração de características e, por fim, classificação ou tomada de decisão a partir dessas características (GONZALEZ; WOODS, 2010). No contexto deste trabalho, a etapa de aquisição corresponde à captura de vídeo pela *webcam*; o pré-processamento e a segmentação envolvem a conversão de formatos de cor e a localização da mão no quadro capturado; e a extração de características corresponde à obtenção dos pontos de referência da mão, detalhada na Seção 2.3. A biblioteca OpenCV, amplamente utilizada em aplicações de visão computacional, oferece as funções necessárias para a captura de vídeo em tempo real, a manipulação de quadros e a exibição de resultados na tela, sendo empregada neste projeto justamente com essa finalidade.

2.3 Detecção de mãos e extração de pontos de referência (*landmarks*)

A detecção de mãos consiste em localizar, dentro de uma imagem, a região correspondente a uma mão humana e, a partir dela, identificar pontos anatômicos de referência — os chamados *landmarks* — que descrevem sua configuração espacial. O MediaPipe Hands, *framework* de código aberto desenvolvido pelo Google, implementa esse processo por meio de um *pipeline* de duas etapas: um detector de palma, responsável por localizar a região da mão na imagem, e um modelo de *landmarks*, responsável por estimar as coordenadas tridimensionais (x, y, z) de 21 pontos correspondentes às articulações e extremidades dos dedos e da palma (ZHANG et al., 2020). Esse *pipeline* foi treinado sobre um amplo conjunto de imagens reais e sintéticas de mãos, o que permite sua utilização com desempenho satisfatório em tempo real, mesmo em *hardware* de uso comum, sem a necessidade de sensores especializados (ZHANG et al., 2020). Por representar cada mão detectada como um

vetor numérico compacto de 21 pontos — em vez de trabalhar diretamente sobre os pixels da imagem —, a extração de *landmarks* tem sido utilizada como etapa intermediária em diversos trabalhos de reconhecimento de línguas de sinais, servindo de entrada para classificadores de aprendizado de máquina treinados para associar configurações da mão a classes específicas, como letras ou palavras (SINCAN; KELES, 2020; SELVARAJ et al., 2022). É essa mesma abordagem — detecção da mão, extração de *landmarks* e posterior classificação — que fundamenta o *pipeline* de reconhecimento proposto neste trabalho.

2.4 Aprendizado de máquina e classificação supervisionada

O aprendizado de máquina é o subcampo da inteligência artificial que estuda algoritmos capazes de identificar padrões em dados e de melhorar seu desempenho em uma tarefa a partir da experiência, sem que suas regras de decisão precisem ser explicitamente programadas (HAYKIN, 2009). Dentre suas categorias, o aprendizado supervisionado — utilizado neste trabalho — consiste em treinar um modelo a partir de um conjunto de dados rotulados, ou seja, exemplos para os quais a resposta correta já é conhecida, de modo que o modelo aprenda a generalizar essa relação entre entradas e saídas para casos ainda não vistos (HAYKIN, 2009). Uma das famílias de algoritmos mais utilizadas nesse tipo de aprendizado são as árvores de decisão, modelos que classificam um exemplo por meio de uma sequência de testes sobre os valores de suas características, em que cada nó interno compara uma variável com um limiar e cada folha corresponde a uma classe. Neste trabalho, os 21 pontos de referência da mão, extraídos conforme descrito na Seção 2.3, compõem o vetor de entradas numéricas utilizado para treinar o classificador responsável por reconhecer cada sinal capturado.

2.5 O classificador Random Forest e a biblioteca Scikit-learn

O *Random Forest* (Floresta Aleatória) é um método de aprendizado de conjunto (*ensemble*) que treina várias árvores de decisão, cada uma sobre uma amostra aleatória dos dados e considerando, em cada divisão, apenas um subconjunto aleatório das variáveis, e define a classe final por votação entre as árvores (BREIMAN, 2001). Essa combinação reduz o risco de sobreajuste (*overfitting*) de uma única árvore, lida bem com relações não lineares entre as variáveis e permite estimar a importância de cada variável para a classificação (BREIMAN, 2001). Neste trabalho,

o *Random Forest* é utilizado para classificar, a partir do vetor de características obtido dos 21 pontos de referência da mão, qual letra do alfabeto ou qual número está sendo representado no momento da captura. Para a implementação do modelo, é utilizada a biblioteca Scikit-learn, biblioteca de código aberto para a linguagem Python que reúne implementações consolidadas de diversos algoritmos de aprendizado de máquina — entre eles o RandomForestClassifier — por meio de uma interface simples e padronizada (PEDREGOSA et al., 2011).

2.6 Trabalhos correlatos

No mercado, destacam-se soluções consolidadas de tradução envolvendo a Libras, como os aplicativos Hand Talk e VLibras, que realizam o caminho inverso ao proposto neste trabalho: a partir de texto ou voz em Língua Portuguesa, geram uma tradução em Libras representada por um avatar tridimensional. Já no meio acadêmico, diversos trabalhos vêm explorando o reconhecimento de línguas de sinais a partir de imagens de câmeras comuns, combinando *frameworks* de detecção de mãos, como o MediaPipe, com classificadores de aprendizado de máquina treinados sobre conjuntos de dados próprios ou públicos — como o AUTSL, conjunto de dados multimodal para a língua de sinais turca (SINCAN; KELES, 2020), e o OpenHands, conjunto de modelos pré-treinados voltados ao reconhecimento de línguas de sinais a partir de poses corporais (SELVARAJ et al., 2022). Esses trabalhos evidenciam que a combinação entre extração de *landmarks* e classificadores supervisionados é uma abordagem viável e já validada na literatura para o reconhecimento de sinais gestuais.

O Quadro 1 sintetiza, de forma comparativa, as principais diferenças técnicas entre as soluções de mercado citadas, o conjunto de modelos OpenHands e a proposta deste trabalho.

Quadro 1 – Comparação técnica entre soluções de tradução de Libras

Aspecto	Hand Talk	VLibras	OpenHands	Tradutor de Libras (este trabalho)

Direção da tradução	Texto/voz → Libras (avatar)	Texto → Libras (avatar)	Postura corporal → Libras (pesquisa)	Webcam (mão) → Texto e Texto → Libras (via VLibras)
Dependência de nuvem	Sim, em grande parte dos recursos	Parcial (há versões locais: desktop/plugin)	Depende da implementação	Parcial: reconhecimento local; avatar do VLibras requer internet
Código aberto	Não (solução comercial)	Sim (software público)	Sim	Sim
Custo	Freemium (recursos pagos)	Gratuito	Gratuito	Gratuito
Bibliotecas/tecnologia base	Proprietária	Ferramentas próprias (LAVID/UFPB)	MediaPipe, modelos de pose pré-treinados	OpenCV, MediaPipe, Scikit-learn, VLibras (avatar)
Escopo linguístico	Libras (sinais e datilologia)	Libras (sinais e datilologia)	Sinais via postura corporal	Alfabeto manual e números (datilologia)
Uso principal	Acessibilidade comercial	Acessibilidade em serviços públicos	Pesquisa em línguas de sinais	Educação, pesquisa e apoio à comunicação

Conforme evidenciado no Quadro 1, o principal diferencial deste trabalho está em reunir, em uma única aplicação web, os dois sentidos da comunicação: de sinais capturados por *webcam* para texto, sentido oposto ao realizado por Hand Talk e VLibras, por meio de um modelo de reconhecimento próprio, cujo processamento de detecção de mãos e classificação ocorre localmente a partir de bibliotecas gratuitas e de código aberto; e de texto para Libras, por meio da integração com o avatar do VLibras, que complementa o sistema sem a necessidade de desenvolver um tradutor próprio, embora dependa de conexão com a internet. Essa característica aproxima o projeto da proposta do OpenHands (SELVARAJ et al., 2022), com a diferença de que este trabalho utiliza um *pipeline* mais simples, baseado em *landmarks* de mãos e em um classificador *Random Forest*, o que favorece sua replicabilidade por outros estudantes e desenvolvedores. O diferencial deste projeto está, portanto, em aplicar essa abordagem ao alfabeto manual e aos números da Libras, com *dataset* próprio e *pipeline* de reconhecimento simples e replicável, construído com bibliotecas gratuitas e de código aberto, podendo servir tanto como ferramenta de apoio à comunicação quanto como material de estudo para novos desenvolvedores.

2.7 Arquitetura de sistemas web

Um sistema web é tipicamente estruturado segundo o modelo cliente-servidor, no qual a camada de *front-end*, executada no navegador do usuário, é responsável pela apresentação da interface e pela interação direta com o usuário, enquanto a camada de *back-end*, executada em um servidor, concentra o processamento de dados, a lógica de negócio e o acesso a recursos computacionais mais exigentes (SOMMERVILLE, 2011). Essa separação de responsabilidades favorece a manutenção do sistema, permite que cada camada evolua de forma independente e possibilita distribuir o processamento de acordo com a capacidade de cada dispositivo (PRESSMAN; MAXIM, 2016).

Neste trabalho, a arquitetura do tradutor de Libras segue esse modelo: o *front-end*, desenvolvido com tecnologias web (HTML5, CSS3 e JavaScript), é responsável por capturar o vídeo da *webcam* do usuário e exibir, em tempo real, o texto traduzido; já o *back-end*, implementado em Python, concentra as etapas mais custosas computacionalmente — a detecção de mãos via MediaPipe, a extração dos 21 pontos de referência (ambas descritas na Seção 2.3) e a classificação das letras e dos números pelo modelo *Random Forest* (Seção 2.5). Essa divisão permite que a interface permaneça responsiva mesmo durante o processamento do reconhecimento de sinais, e concentra as atualizações do modelo de aprendizado de máquina em um único ponto do sistema, facilitando sua manutenção e evolução futura. Para a tradução de texto em Libras, o *front-end* incorpora ainda o VLibras, que apresenta o avatar tridimensional responsável por sinalizar o texto digitado. O detalhamento da implementação dessa arquitetura é apresentado no Capítulo 3.

2.8 Segurança e privacidade de dados em sistemas de visão computacional

Sistemas de visão computacional que operam a partir de imagens de *webcam* lidam, por natureza, com dados sensíveis, uma vez que capturam informações visuais do usuário em tempo real. A Lei Geral de Proteção de Dados Pessoais (LGPD), Lei nº 13.709, de 14 de agosto de 2018, estabelece que o tratamento de dados pessoais — categoria que pode incluir imagens capazes de identificar uma pessoa — deve observar princípios como finalidade específica, necessidade e transparência, além de exigir, em regra, o consentimento do titular dos dados (BRASIL, 2018). Do ponto de vista técnico, a literatura de segurança da informação recomenda que a transmissão

de dados entre cliente e servidor seja protegida por criptografia, de modo a impedir a interceptação de informações sensíveis durante o tráfego na rede (STALLINGS, 2015).

Neste trabalho, a captura de vídeo é processada durante a sessão do usuário, e apenas os pontos de referência da mão — e não os quadros de imagem em si — são utilizados pelo classificador, o que reduz a quantidade de dados sensíveis manipulados pelo sistema. Ainda assim, para uma eventual implantação em produção, recomenda-se que nenhuma imagem de vídeo seja armazenada em servidor após o processamento, que toda comunicação entre *front-end* e *back-end* seja realizada de forma segura, e que o usuário seja informado, de forma clara, sobre a captura e o uso de sua imagem antes de iniciar a sessão, em conformidade com os princípios estabelecidos pela LGPD (BRASIL, 2018). Essas considerações são retomadas, com mais detalhe, no Capítulo 3, por ocasião da descrição da implementação do sistema.

3 DESENVOLVIMENTO

Este capítulo apresenta o desenvolvimento do tradutor de Libras, abordando os requisitos levantados para o sistema, o ambiente e as ferramentas de desenvolvimento, o detalhamento da arquitetura já introduzida na Seção 2.7, o processo de construção do *dataset* de sinais, o treinamento do classificador, a implementação da interface web e a integração com o VLibras para a tradução de texto em Libras. Os resultados obtidos a partir da execução dessas etapas são apresentados no Capítulo 4.

3.1 Requisitos do sistema

A definição dos requisitos do tradutor de Libras parte dos objetivos específicos apresentados na Seção 1.5. Os requisitos funcionais (RF) descrevem as funcionalidades que o sistema deve oferecer, enquanto os requisitos não funcionais (RNF) estabelecem restrições e qualidades esperadas de seu funcionamento.

Requisitos funcionais:

RF01 – O sistema deve capturar, em tempo real, o vídeo da *webcam* do usuário.

RF02 – O sistema deve detectar a mão do usuário no quadro capturado e extrair os 21 pontos de referência correspondentes.

RF03 – O sistema deve classificar os pontos de referência extraídos, identificando a letra do alfabeto manual da Libras ou o número representado, conforme o modo selecionado (letras ou números).

RF04 – O sistema deve montar palavras e frases automaticamente a partir da sequência de letras e números reconhecidos.

RF05 – O sistema deve permitir a inserção de espaço entre as palavras, inclusive de forma mãos-livres, ao abaixar a mão, e a exclusão do último caractere inserido.

RF06 – O sistema deve exibir, em tempo real, o texto traduzido na interface web.

RF07 – O sistema deve permitir concluir a frase, ouvi-la por síntese de voz, copiá-la e mantê-la em um histórico de frases traduzidas, que pode ser baixado em arquivo de texto (.txt).

RF08 – O sistema deve oferecer a tradução de texto para Libras, apresentando o texto digitado a um avatar tridimensional que o sinaliza, por meio da integração com o VLibras.

Requisitos não funcionais:

RNF01 – O sistema deve responder em tempo real, com baixa latência entre a captura do gesto e a exibição do texto correspondente.

RNF02 – O sistema deve funcionar com uma *webcam* comum, sem exigir sensores ou *hardware* especializado.

RNF03 – O sistema deve ser construído exclusivamente com bibliotecas gratuitas e de código aberto, conforme discutido na Seção 1.3.

RNF04 – O sistema deve adotar comunicação segura entre *front-end* e *back-end*, conforme as diretrizes apresentadas na Seção 2.8.

3.2 Ambiente e ferramentas de desenvolvimento

O desenvolvimento do sistema utiliza a linguagem Python como núcleo do processamento de visão computacional e aprendizado de máquina, em conjunto com as bibliotecas já apresentadas no Capítulo 2: OpenCV, para a captura e manipulação de vídeo (Seção 2.2); MediaPipe (HandLandmarker), para a detecção de mãos e extração dos 21 pontos de referência (Seção 2.3), com o modelo hand_landmarker.task, baixado automaticamente na primeira execução; e Scikit-learn, para o treinamento do classificador *Random Forest* (Seção 2.5). Para a camada de *back-end*, é utilizado o microframework Flask, responsável por expor os *endpoints* que recebem os quadros de vídeo do *front-end* (ou capturam o vídeo diretamente da câmera do servidor), executam o *pipeline* de reconhecimento e retornam o caractere classificado (letra ou número) em formato JSON. A interface web (*front-end*) é construída com HTML5, CSS3 e JavaScript, conforme descrito na Seção 2.7, sendo responsável pela captura do vídeo da *webcam* por meio da API de mídia do navegador (quando a câmera do navegador é escolhida) e pela exibição do texto traduzido. Para a tradução de texto em Libras, é integrado o VLibras, conjunto de ferramentas públicas e gratuitas desenvolvido pelo Laboratório de Aplicações de Vídeo Digital (LAVID) da UFPB, que disponibiliza um avatar tridimensional capaz de sinalizar o texto; esse recurso é incorporado ao *front-end* como um *widget* e requer conexão com a internet.

Como ambiente de desenvolvimento, é utilizada uma IDE Python, com controle de versão do código por meio do Git.

3.3 Arquitetura do sistema

A arquitetura do sistema segue o modelo cliente-servidor apresentado na Seção 2.7. O *front-end* é responsável por: (a) exibir a interface e permitir a escolha da fonte de vídeo, que pode ser a câmera do navegador (HTML5), da qual os quadros são capturados em intervalos regulares e enviados ao servidor como imagens codificadas, ou uma câmera conectada ao servidor, selecionada por índice (webcam padrão, câmeras virtuais ou USB); (b) consultar o estado do reconhecimento e atualizar na tela o caractere, a confiança e a frase; (c) enviar ao servidor os comandos de edição (espaço, apagar, concluir, limpar e troca de modo); e (d) incorporar o VLibras para a tradução de texto em Libras. O *back-end*, implementado em Flask, expõe *endpoints* para receber os quadros do navegador, transmitir o vídeo com os pontos da mão sobrepostos, informar o estado atual do reconhecimento, receber os comandos de edição e exportar o histórico de frases em arquivo .txt. Quando a câmera do servidor é utilizada, a captura é feita pelo OpenCV em uma *thread* dedicada, o que evita que a leitura do vídeo bloqueie o atendimento das requisições e permite a reconexão automática da câmera. Em ambos os casos o *pipeline* de reconhecimento é o mesmo — detecção de mão, extração de *landmarks*, normalização e classificação pelo *Random Forest* — e o resultado é retornado em formato JSON, com o caractere reconhecido, o nível de confiança e o progresso da confirmação. Essa separação permite que o processamento mais pesado fique concentrado no servidor, mantendo o *front-end* leve e responsivo, conforme já discutido na Seção 2.7.

3.4 Construção do *dataset*

A construção do *dataset* próprio de sinais segue a abordagem descrita na Seção 2.3: para cada captura, a imagem da mão é processada pelo MediaPipe Hands, que retorna os 21 pontos de referência (coordenadas x, y e z), com confiança mínima de 0,6 para a detecção, a presença e o rastreamento da mão. Esses pontos são normalizados: o pulso é adotado como origem e todas as coordenadas são divididas pela distância entre o pulso e a base do dedo médio, o que torna o vetor invariante à posição da mão na tela e à distância até a câmera. A rotação da mão não é removida, pois a orientação da palma faz parte do significado de sinais com configurações de

dedos parecidas, como K, H e P. A esses 63 valores somam-se outros 63, que representam a variação de cada coordenada em relação ao quadro anterior (velocidade), totalizando 126 características por amostra e auxiliando a distinguir letras com pequeno movimento (H, J, K, X e Z) de letras estáticas semelhantes. A mesma rotina de extração e normalização é compartilhada entre o *script* de coleta e o servidor, garantindo que o modelo receba, na predição, o mesmo tipo de vetor usado no treinamento. A coleta é feita por um *script* próprio, que exibe o vídeo da *webcam* com os pontos da mão sobrepostos e permite selecionar, pelo teclado, a letra (A a Z) ou o número (0 a 9) a ser gravado. A gravação pode ser manual, com uma amostra a cada acionamento, ou contínua, em que cada quadro processado gera uma amostra; o modo contínuo é indicado para as letras que envolvem movimento (H, J, K, X e Z), pois permite registrar o gesto completo em vários quadros. Para favorecer a generalização do modelo, as amostras são coletadas com mãos de diferentes pessoas. O *dataset* resultante contém amostras das 26 letras e dos dez números (0 a 9), com 8.839 amostras no total (7.537 de letras e 1.302 de números), em quantidades que variam de 101 (número 3) a 720 (letra H) por classe, conforme a Figura 6 do Capítulo 4. Cada amostra coletada é rotulada com a classe correspondente (a letra ou o número) e armazenada em um arquivo estruturado no formato CSV, no qual cada linha representa uma amostra e cada coluna corresponde a uma das 126 características.

3.5 Pré-processamento e treinamento do classificador

Os vetores de características chegam ao treinamento já normalizados pela etapa de extração descrita na Seção 3.4. Em seguida, o *dataset* é dividido em um conjunto de treino e um conjunto de teste (20% das amostras), destinado à validação da capacidade de generalização do modelo. O classificador *Random Forest* (RandomForestClassifier) é então treinado sobre o conjunto de treino, construindo um conjunto de árvores de decisão cujas respostas são combinadas por votação, conforme descrito na Seção 2.5. Antes da avaliação final, a capacidade de generalização é estimada por validação cruzada de cinco partições (*5-fold*) sobre o conjunto de treino. Após o treinamento, o desempenho do modelo é avaliado sobre o conjunto de teste, formado por dados nunca vistos pelo modelo, e os resultados dessa avaliação — incluindo a acurácia, a precisão, a revocação e o *F1-score* de cada classe — são apresentados no Capítulo 4. Por fim, o modelo treinado é salvo, junto com o

codificador de rótulos, em um arquivo (.pkl) gravado com a biblioteca joblib, para ser carregado pelo *back-end* durante o reconhecimento.

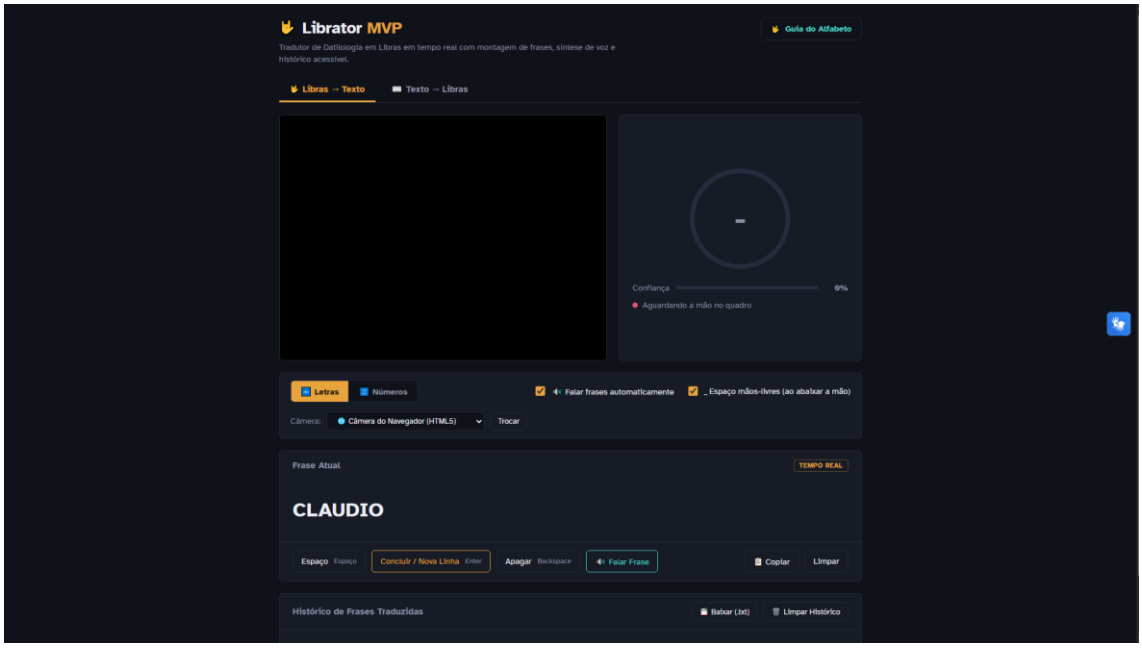
No reconhecimento em tempo real, o servidor carrega o modelo treinado e aplica, a cada quadro, uma sequência de regras para decidir se um caractere deve ser adicionado à frase. Primeiro, o *Random Forest* calcula a probabilidade de cada classe, e o sistema considera apenas as classes do modo selecionado (Letras ou Números), renormalizando suas probabilidades; isso evita confundir sinais semelhantes de grupos diferentes, como V e 2 ou O e 0. Se a probabilidade somada do grupo for inferior a 0,25, o sinal é descartado, por não se parecer com nenhum sinal do modo. Em seguida, só são aceitas as predições com confiança igual ou superior a 0,65. As oito últimas predições aceitas são comparadas por votação, e o caractere mais frequente só se torna candidato se obtiver pelo menos metade dos votos. Por fim, o candidato precisa ser mantido de forma estável por 1 segundo, período indicado por uma barra de progresso sobre o vídeo, para ser confirmado e adicionado à frase. Após a confirmação, o mesmo caractere só é aceito novamente se o usuário trocar de sinal ou retirar a mão do quadro. Esses parâmetros reduzem a inserção de caracteres incorretos causada por quadros ambíguos ou por transições entre gestos, à custa de um atraso de cerca de um segundo para a confirmação de cada caractere.

3.6 Implementação da interface web e montagem de palavras

A interface web é organizada em duas abas: Libras → Texto e Texto → Libras. Na aba Libras → Texto, a tela exibe o vídeo capturado pela *webcam*, com os pontos de referência da mão sobrepostos, e, ao lado, um painel com o caractere reconhecido e, logo abaixo dele, a barra com o nível de confiança da predição e uma mensagem de status, como "Aguardando a mão no quadro". O usuário pode alternar entre os modos Letras e Números, escolher a câmera utilizada (a câmera do navegador ou uma câmera do servidor, selecionada por índice) e ativar ou desativar a leitura automática das frases e o espaço mãos-livres, acionado ao abaixar a mão. A cada quadro processado, o *back-end* retorna o caractere candidato, o nível de confiança e o progresso da confirmação, conforme as regras descritas na Seção 3.5. Os caracteres confirmados são concatenados na frase atual, exibida em destaque e em tempo real. Abaixo dela, botões com atalhos de teclado permitem inserir espaço,

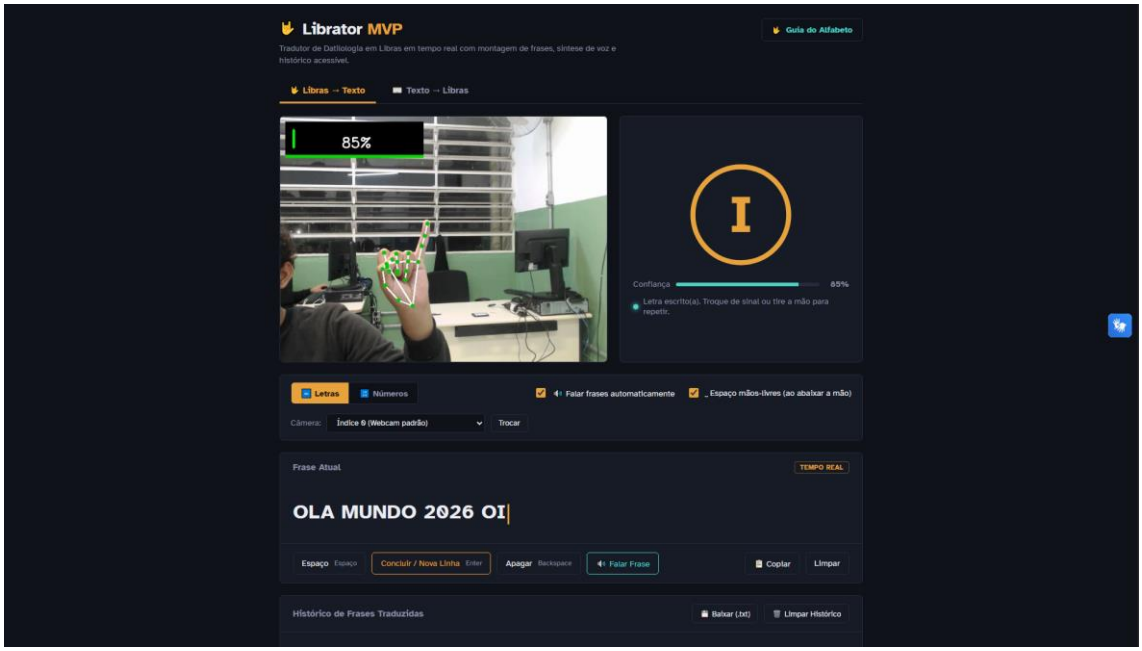
concluir a frase e iniciar uma nova linha, apagar o último caractere (RF05), ouvir a frase por síntese de voz, copiá-la e limpá-la. As frases concluídas são armazenadas em um histórico de frases traduzidas, que pode ser baixado em arquivo de texto (.txt) ou limpo (RF07). Um botão no cabeçalho dá acesso ao guia do alfabeto, que auxilia o usuário na realização dos sinais. As Figuras 2 e 3 apresentam as telas do sistema.

Figura 2 – Tela inicial do sistema (aba Libras → Texto)



Fonte: Captura de tela do sistema desenvolvido (2026).

Figura 3 – Reconhecimento de sinais em tempo real e montagem da frase



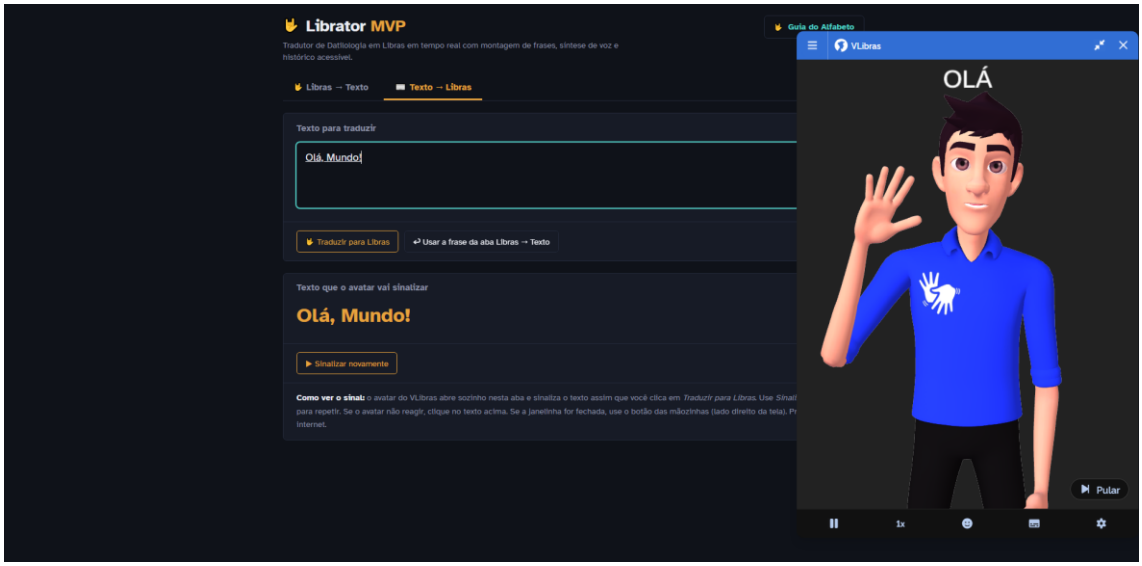
Fonte: Captura de tela do sistema desenvolvido (2026).

A Figura 2 mostra a tela inicial, em que o sistema aguarda a mão do usuário no quadro e o nível de confiança permanece em 0%. A Figura 3 mostra o sistema em funcionamento: a mão é detectada, os pontos de referência são desenhados sobre o vídeo, a letra "I" é exibida no painel lateral com 85% de confiança e a frase atual é atualizada com os caracteres aceitos.

3.7 Integração com o VLibras para tradução de texto em Libras

Para o sentido texto → Libras, a aba Texto → Libras oferece um campo para a digitação do texto, o botão "Traduzir para Libras" e o botão "Usar a frase da aba Libras → Texto", que reaproveita a frase montada pelo reconhecimento de sinais. O texto a ser sinalizado é exibido em destaque e o avatar do VLibras abre na própria página e sinaliza o texto assim que o usuário aciona a tradução; o botão "Sinalizar novamente" repete o sinal (RF08). Como a tradução é realizada pelo VLibras, serviço externo ao sistema, esse recurso depende de conexão com a internet, diferentemente do reconhecimento de sinais, que é executado localmente. A Figura 4 apresenta essa aba com o avatar sinalizando o texto "Olá, Mundo!".

Figura 4 – Aba Texto → Libras com o avatar do VLibras



Fonte: Captura de tela do sistema desenvolvido (2026).

3.8 Considerações de segurança na implementação

Em conformidade com os princípios discutidos na Seção 2.8, nenhum quadro de vídeo é armazenado em disco: os quadros recebidos são decodificados em memória, processados pelo *pipeline* de reconhecimento e descartados, e apenas os

pontos de referência extraídos e os caracteres reconhecidos são mantidos temporariamente em memória durante a execução. Na versão atual, a comunicação entre o *front-end* e o *back-end* em Flask ocorre por HTTP em rede local, sem criptografia, o que é adequado ao ambiente de desenvolvimento e de testes, mas deve ser substituído por HTTPS em uma implantação em produção. Quando a câmera do navegador é utilizada, a interface solicita explicitamente a permissão do usuário para acessar a *webcam*, conforme exigido pelos navegadores modernos, reforçando a transparência quanto ao uso da imagem capturada.

4 RESULTADOS

Este capítulo apresenta os dados e as evidências obtidos após a implementação e os testes do tradutor de Libras. Os resultados são expostos de forma objetiva, por meio de quadros, métricas e capturas de tela, e a interpretação desses dados é feita na Seção 4.4. Inicialmente, são relacionadas as funcionalidades entregues em comparação com os requisitos definidos no Capítulo 3; em seguida, são apresentados os resultados dos testes e o retorno obtido com o avaliador externo.

4.1 Funcionalidades implementadas

O Quadro 2 relaciona cada requisito funcional (RF) e não funcional (RNF) definido na Seção 3.1 com a sua situação ao final do desenvolvimento, classificada como atendido integralmente, atendido parcialmente ou não implementado, acompanhada da respectiva justificativa.

Quadro 2 – Situação dos requisitos do sistema

Req.	Descrição	Situação	Justificativa
RF01	Capturar, em tempo real, o vídeo da *webcam* do usuário.	Atendido	Vídeo da câmera exibido na interface (Figura 3).
RF02	Detectar a mão e extrair os 21 pontos de referência.	Atendido	Pontos de referência desenhados sobre a mão (Figura 3).
RF03	Classificar os pontos extraídos e identificar a letra ou o número representado.	Atendido	Letra "I" reconhecida, com 85% de confiança exibida abaixo dela (Figura 3); números na frase da Figura 5; acurácia de 99% no conjunto de teste (Figura 7).
RF04	Montar palavras e frases automaticamente a partir dos caracteres reconhecidos.	Atendido	Frase "OLA MUNDO 2026 OI" montada automaticamente (Figura 5).
RF05	Inserir espaço (inclusive mãos-livres) e excluir o último caractere.	Atendido	Comandos de espaço e apagar, com botões e atalhos, e espaço mãos-livres; aprovados no caso CT06 (Quadro 3).
RF06	Exibir o texto traduzido em tempo real na interface web.	Atendido	Frase atual atualizada em tempo real (Figuras 3 e 5).
RF07	Concluir, falar (síntese de voz), copiar e guardar frases em histórico baixável.	Atendido	Frase concluída vai ao histórico, que pode ser baixado em .txt; leitura por voz disponível na interface (Figura 3).
RF08	Traduzir texto para Libras por meio do avatar do VLibras.	Atendido	Avatar sinalizando o texto "Olá, Mundo!" (Figura 4).

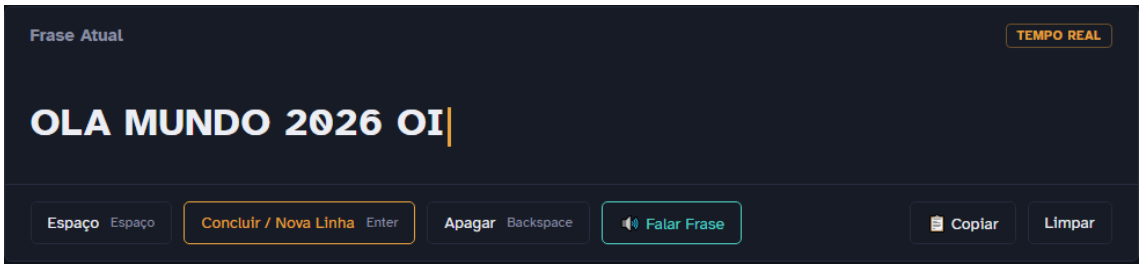
Req.	Descrição	Situação	Justificativa
RNF01	Responder em tempo real, com baixa latência.	Atendido	Processamento de 73,9 ms por quadro com a mão visível (Quadro 4); caractere confirmado após 1 s de permanência estável, por projeto.
RNF02	Funcionar com uma *webcam* comum, sem *hardware* especializado.	Atendido	Funciona com a webcam padrão, com a câmera do navegador ou com câmeras virtuais (Figura 3).
RNF03	Utilizar exclusivamente bibliotecas gratuitas e de código aberto.	Atendido	Python, OpenCV, MediaPipe, Scikit-learn e Flask são de código aberto; o VLibras é software público.
RNF04	Adotar comunicação segura entre *front-end* e *back-end*.	Parcial	Nenhum quadro é gravado em disco, mas a comunicação usa HTTP em rede local, sem HTTPS (Seção 3.8).

Fonte: Elaborado pelos autores (2026).

Do total de 12 requisitos definidos, 11 foram atendidos integralmente e 1 (RNF04) foi atendido parcialmente; nenhum deixou de ser implementado. O RNF04 foi classificado como parcial porque a comunicação entre o *front-end* e o *back-end* ocorre por HTTP em rede local, sem criptografia, e a adoção de HTTPS é indicada para uma implantação em produção.

As Figuras 3 e 4, apresentadas no Capítulo 3, mostram o sistema em funcionamento nos dois sentidos de tradução: o reconhecimento de uma letra a partir da *webcam* e a sinalização de um texto pelo avatar do VLibras. A Figura 5 mostra a frase "OLA MUNDO 2026 OI", montada pelo sistema a partir de letras e números reconhecidos, com os espaços inseridos entre as palavras e os controles de edição disponíveis logo abaixo.

Figura 5 – Frase montada pelo sistema com letras e números



Fonte: Captura de tela do sistema desenvolvido (2026).

4.2 Resultados dos testes

Os testes funcionais foram conduzidos sobre as principais rotinas do sistema, conforme previsto no objetivo específico (f) da Seção 1.5. O Quadro 3 apresenta os casos de teste executados, o resultado esperado e o resultado obtido em cada um deles.

Quadro 3 – Casos de teste funcionais

ID	Req.	Procedimento	Resultado esperado	Resultado obtido / situação
CT01	RF01	Abrir a página e permitir o acesso à *webcam*.	Vídeo da câmera exibido na tela.	Aprovado
CT02	RF02	Posicionar a mão diante da câmera.	21 pontos de referência detectados.	Aprovado
CT03	RF03	Realizar o sinal de cada letra do alfabeto.	Letra correta exibida.	Aprovado
CT04	RF03	Alternar para o modo Números e realizar o sinal de cada número do *dataset*.	Número correto exibido.	Aprovado
CT05	RF04	Sinalizar uma sequência de letras e números.	Frase formada na ordem correta.	Aprovado
CT06	RF05	Inserir espaço (botão e mãos-livres) e apagar o último caractere.	Espaço inserido; último caractere removido.	Aprovado
CT07	RF07	Concluir uma frase com a leitura automática ativa.	Frase falada e registrada no histórico.	Aprovado
CT08	RF08	Digitar um texto e acionar "Traduzir para Libras".	Avatar do VLibras sinaliza o texto.	Aprovado
CT09	RNF01	Medir o tempo de processamento de cada quadro com a mão visível.	Processamento compatível com o uso em tempo real.	Aprovado: 73,9 ms por quadro, em média (Quadro 4).
CT10	RNF04	Iniciar a captura sem conceder a permissão.	Captura bloqueada até a autorização.	Aprovado

Fonte: Elaborado pelos autores (2026).

Dos 10 casos de teste executados, os 10 foram aprovados, o que corresponde a 100% de aprovação. O Quadro 4 reúne as métricas de desempenho do classificador e do sistema em execução.

Quadro 4 – Métricas de desempenho do classificador e do sistema

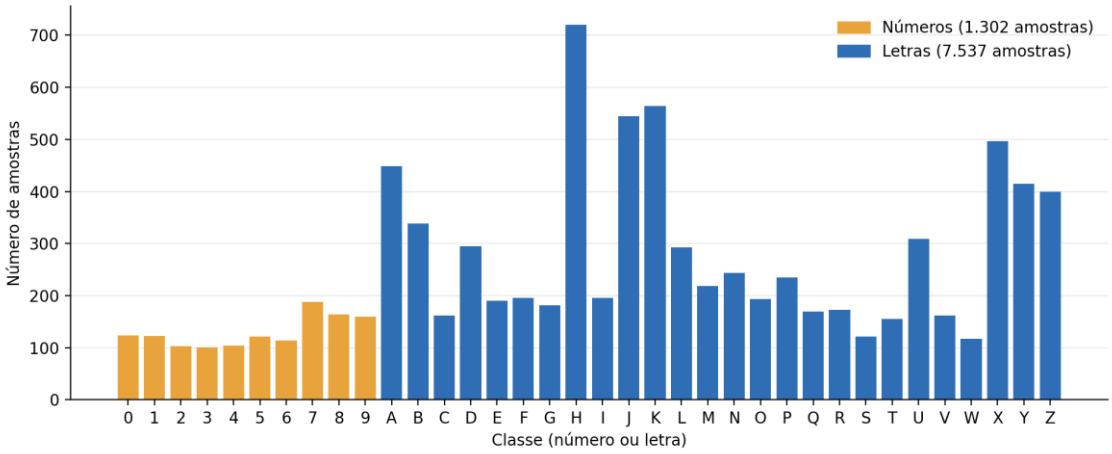
Métrica	Valor obtido
Algoritmo de classificação	*Random Forest* (RandomForestClassifier)
Total de amostras do *dataset*	8.839 (7.537 de letras e 1.302 de números)
Amostras por classe	De 101 (número 3) a 720 (letra H)

Métrica	Valor obtido
Características por amostra	126 (63 de posição normalizada + 63 de variação em relação ao quadro anterior)
Amostras no conjunto de teste	1.768 (20%)
Acurácia no conjunto de teste	99%
Acurácia média na validação cruzada (5 partições, treino)	98,57% (± 0,23 p.p.)
Precisão, revocação e *F1-score* médios (macro)	0,99 / 0,99 / 0,99
Classes com menor desempenho	R (revocação 0,94); 9 (precisão 0,94); J (revocação 0,95)
Limiar mínimo de confiança da predição	0,65
Probabilidade mínima do grupo do modo (Letras ou Números)	0,25
Suavização: janela de predições e proporção mínima de votos	8 predições; 50% dos votos
Tempo de permanência estável para confirmar um caractere	1 s
Taxa de quadros do *script* de coleta (captura + MediaPipe), em FPS	30,6 em média (mín. 12,3; máx. 54,3), em 717 quadros
Latência de processamento por quadro, com mão visível (1.000 quadros)	73,9 ms em média (mediana 71,9; p95 85,7; máx. 101,3)
Latência de processamento por quadro, sem mão (1.000 quadros)	21,6 ms em média (mediana 20,3; p95 29,9; máx. 51,3)

Fonte: Elaborado pelos autores (2026).

A Figura 6 mostra a distribuição das amostras por classe no *dataset* atual, que reúne 8.839 amostras. O número de amostras varia de 101 (número 3) a 720 (letra H); as letras com movimento (H, J, K, X e Z), além de A e Y, estão entre as classes com mais amostras, enquanto cada número tem entre 101 e 188.

Figura 6 – Distribuição das amostras por classe no dataset



Fonte: Elaborado pelos autores (2026).

A Figura 7 apresenta o relatório de classificação gerado ao final do treinamento, com a precisão, a revocação e o *F1-score* de cada classe, calculados sobre 1.768 amostras do conjunto de teste, que não foram utilizadas no treinamento. A acurácia global foi de 99% e a acurácia média na validação cruzada de cinco partições, calculada sobre o conjunto de treino, foi de 98,57%. Todos os números, incluindo o 7, obtiveram revocação de 1,00, e a precisão foi de 1,00, exceto para o número 9 (0,94). Entre as letras, os menores valores foram observados na revocação de R (0,94) e de J (0,95); nas demais classes, precisão e revocação ficaram entre 0,97 e 1,00.

Figura 7 – Relatório de classificação do modelo no conjunto de teste

```
Acurácia média (5-fold CV, conjunto de treino): 0.9857 (+/- 0.0023)

=== Relatório de Classificação (conjunto de teste, dados nunca vistos) ===
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        25
           1       1.00      1.00      1.00        24
           2       1.00      1.00      1.00        20
           3       1.00      1.00      1.00        20
           4       1.00      1.00      1.00        21
           5       1.00      1.00      1.00        24
           6       1.00      1.00      1.00        23
           7       1.00      1.00      1.00        38
           8       1.00      1.00      1.00        33
           9       0.94      1.00      0.97        32
           A       1.00      1.00      1.00        90
           B       1.00      1.00      1.00        68
           C       1.00      1.00      1.00        32
           D       0.97      1.00      0.98        59
           E       1.00      1.00      1.00        38
           F       1.00      0.97      0.99        39
           G       1.00      0.97      0.99        36
           H       0.99      0.99      0.99       144
           I       0.97      1.00      0.99        39
           J       1.00      0.95      0.98       109
           K       0.99      1.00      1.00       113
           L       1.00      1.00      1.00        59
           M       1.00      1.00      1.00        44
           N       1.00      1.00      1.00        49
           O       1.00      1.00      1.00        39
           P       1.00      1.00      1.00        47
           Q       1.00      1.00      1.00        34
           R       1.00      0.94      0.97        35
           S       1.00      1.00      1.00        24
           T       0.97      1.00      0.98        31
           U       0.98      1.00      0.99        62
           V       1.00      0.97      0.98        32
           W       1.00      1.00      1.00        23
           X       1.00      1.00      1.00        99
           Y       0.98      1.00      0.99        83
           Z       0.99      1.00      0.99        80

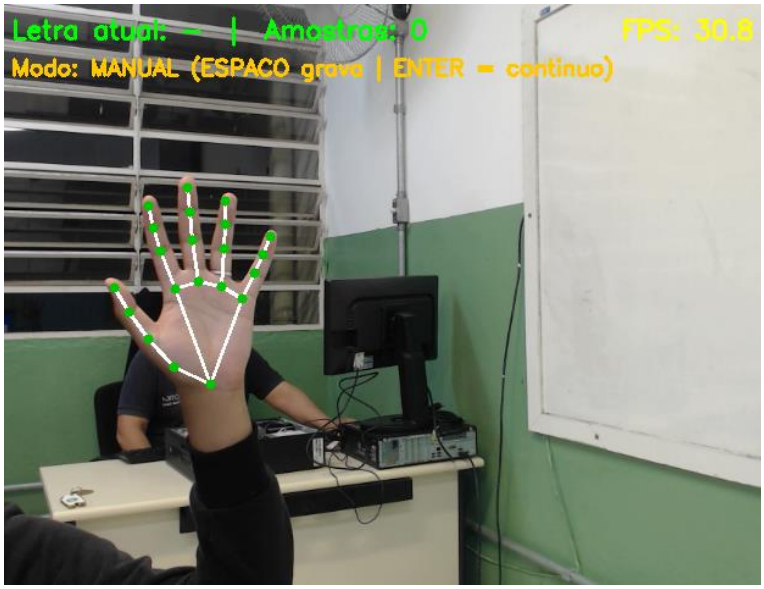
    accuracy                           0.99      1768
   macro avg       0.99      0.99      0.99      1768
weighted avg       0.99      0.99      0.99      1768


Top 10 features mais importantes:
      y8: 0.0400
     y12: 0.0395
     y11: 0.0380
      y7: 0.0371
     y16: 0.0358
     y15: 0.0348
      y4: 0.0347
     y20: 0.0332
     y19: 0.0304
      y3: 0.0293
```

Fonte: Saída do script de treinamento do modelo (2026).

Quanto ao desempenho durante a execução, a Figura 8 mostra o contador de FPS exibido pelo *script* de coleta de dados, que registrou 30,8 quadros por segundo no momento da captura. Em uma coleta de 717 quadros, a média foi de 30,6 FPS, com mínimo de 12,3 e máximo de 54,3. No servidor, o processamento de um quadro com a mão visível levou, em média, 73,9 ms (mediana de 71,9 ms; percentil 95 de 85,7 ms; máximo de 101,3 ms), contra 21,6 ms em quadros sem mão, medidos em 1.000 quadros de cada tipo.

Figura 8 – Contador de FPS durante a captura da mão



Fonte: Captura de tela do script de coleta de dados (2026).

Na execução dos casos de teste do Quadro 3 não foram encontradas falhas. O Quadro 5 registra o único problema relevante, identificado durante o desenvolvimento e já corrigido.

Quadro 5 – Problema identificado durante o desenvolvimento e correção aplicada

ID	Falha observada	Causa identificada	Correção aplicada
F01	Dificuldade no reconhecimento de algumas letras do alfabeto	Pouca variedade de mãos no *dataset* inicial, com variação entre mãos de pessoas diferentes	Ampliação do *dataset* com amostras de mãos diferentes, o que facilitou o reconhecimento

Fonte: Elaborado pelos autores (2026).

4.3 Avaliação do sistema por avaliador externo

Para complementar os testes funcionais, o sistema foi testado por uma pessoa que não faz parte do grupo de desenvolvimento: o professor orientador do trabalho, que também leciona a disciplina de DTCC. O Quadro 6 apresenta o retorno obtido em

cada critério avaliado. O avaliador aprovou todos os critérios. Por se tratar de um único avaliador, o resultado tem caráter qualitativo e representa uma primeira impressão de uso, e não uma pesquisa de satisfação com amostra representativa.

Quadro 6 – Retorno do avaliador externo que testou o sistema

Critério	Pergunta	Resultado
Facilidade de uso	A interface foi fácil de utilizar?	Aprovado
Reconhecimento de letras	As letras foram reconhecidas corretamente?	Aprovado
Reconhecimento de números	Os números foram reconhecidos corretamente?	Aprovado
Velocidade	A resposta do sistema foi rápida?	Aprovado
Tradução texto → Libras	O avatar do VLibras sinalizou o texto de forma compreensível?	Aprovado
Clareza da interface	O texto traduzido foi fácil de visualizar?	Aprovado

Fonte: Elaborado pelos autores (2026).

O avaliador considerou que o sistema está adequado na versão atual e não apresentou sugestões de alteração. Como a avaliação foi feita por uma única pessoa externa ao grupo, sem a participação de pessoas surdas, esse retorno não substitui uma avaliação com um número maior de participantes, indicada como trabalho futuro.

4.4 Discussão

Os resultados indicam que o sistema atendeu integralmente a 11 dos 12 requisitos definidos e parcialmente ao restante (RNF04), com 100% de aprovação nos testes funcionais. O classificador *Random Forest* alcançou 99% de acurácia no conjunto de teste e 98,57% na validação cruzada, e o processamento de um quadro com a mão visível levou, em média, 73,9 ms no servidor (cerca de 13 quadros por segundo), o que se mostrou suficiente para o uso em tempo real. O custo mais de três vezes maior em quadros com mão (73,9 ms contra 21,6 ms sem mão) indica que a extração dos pontos de referência e a classificação respondem pelo acréscimo. Esses números sugerem que a combinação entre a extração dos 21 pontos de referência da mão pelo MediaPipe e um classificador baseado em árvores, abordagem de reconhecimento a partir de *landmarks* já validada na literatura (SINCAN; KELES, 2020; SELVARAJ et al., 2022), mostrou-se adequada para reconhecer letras e números da Libras em tempo real com uma *webcam* comum. Além disso, a integração com o VLibras permitiu que o sistema cobrisse também o sentido inverso, de texto

para Libras, sem a necessidade de desenvolver um tradutor próprio. Dessa forma, o objetivo geral do trabalho foi atingido, e a questão de pesquisa apresentada na Seção 1.4 pode ser respondida de forma afirmativa.

A análise do relatório de classificação mostra que os erros no conjunto de teste foram poucos e se concentraram na revocação das letras R (0,94) e J (0,95) e na precisão do número 9 (0,94), o que é compatível com a similaridade visual entre algumas configurações de mão, já apontada na Seção 2.1. Nos testes de uso do sistema, porém, não foram observadas confusões perceptíveis entre as letras. As letras que envolvem movimento (H, J, K, X e Z) obtiveram *F1-score* entre 0,98 e 1,00 e funcionaram sem problemas durante o uso, o que indica que a gravação contínua do gesto completo durante a coleta, aliada às características de velocidade, foi suficiente para o modelo aprender seus padrões. No início do desenvolvimento, o reconhecimento de algumas letras do alfabeto apresentou dificuldades, causadas pela pouca variedade de mãos no *dataset* inicial; ao longo do projeto, foram gravados mais tipos de mão, o que reduziu os erros de identificação e favoreceu a generalização do modelo. O limiar de confiança de 0,65, a votação entre as oito últimas predições e a exigência de um segundo de permanência estável funcionam como filtros: transições entre gestos e quadros ambíguos, que poderiam gerar erros pontuais como os observados em R, J e 9, dificilmente se mantêm por tempo suficiente para serem confirmados, o que ajuda a explicar a ausência de confusões perceptíveis durante o uso. A baixa precisão do número 9 indica que sinais de letras foram classificados como 9, já que todos os números tiveram revocação de 1,00; no sistema em execução, a separação entre os modos Letras e Números impede esse tipo de confusão, pois apenas as classes do modo selecionado são consideradas.

Entre as limitações observadas, destacam-se: o reconhecimento baseado na configuração da mão e em sua variação em relação ao quadro anterior, sem modelagem de sequências mais longas de movimento, o que impede a tradução de palavras e frases em Libras; a dependência de conexão com a internet e de um serviço de terceiros para a tradução de texto em Libras, cujo vocabulário e funcionamento não são controlados pelos autores; o atraso de cerca de um segundo para confirmar cada caractere, adotado para reduzir erros, mas que torna a digitação mais lenta; o custo de processamento de cerca de 74 ms por quadro com a mão visível, que limita o reconhecimento a aproximadamente 13 quadros por segundo no servidor, abaixo da

taxa de captura da câmera, de cerca de 30 quadros por segundo; a comunicação por HTTP, sem criptografia, adequada ao ambiente local de testes; e o desbalanceamento do *dataset*, em que as classes de letras com movimento têm mais amostras (por exemplo, 720 amostras para a letra H) do que os números (de 101 a 188 amostras por número). Além disso, como o conjunto de teste foi obtido por divisão aleatória das amostras, quadros muito parecidos de um mesmo gesto podem ter ficado tanto no treino quanto no teste, o que tende a tornar a acurácia de 99% otimista em relação ao uso com pessoas que não participaram da coleta. Há ainda uma limitação de alcance: a distância máxima de uso foi de cerca de 1 metro, pois, acima disso, a mão deixa de ser detectada ou é reconhecida de forma errada, já que, dependendo da câmera, os pontos da mão não são identificados corretamente. Observou-se também que algumas câmeras não são ideais para o uso, pois dificultam a leitura da mão, o que ajuda a explicar a variação de desempenho entre equipamentos. Em contrapartida, o avaliador externo que testou o sistema o considerou adequado, sem sugestões de alteração, embora uma única avaliação, feita sem a participação de pessoas surdas, não permita generalizar esse resultado.

Essas limitações apontam direções para trabalhos futuros, como a avaliação do modelo com a separação das amostras por pessoa, a ampliação e a diversificação do *dataset*, o uso de modelos sequenciais, que considerem vários quadros consecutivos, para reconhecer sinais dinâmicos, a melhoria do reconhecimento a maiores distâncias, o uso das duas mãos e a realização de testes com um número maior de usuários, incluindo pessoas surdas.

5 CONCLUSÕES

Este trabalho teve como objetivo desenvolver um sistema web capaz de reconhecer, em tempo real e por meio de uma *webcam* comum, letras e números da Língua Brasileira de Sinais (Libras) e convertê-los em texto, além de traduzir texto em Libras por meio do avatar do VLibras. Com base nos resultados apresentados no Capítulo 4, conclui-se que o objetivo geral foi atingido e que a questão de pesquisa apresentada na Seção 1.4 pode ser respondida de forma afirmativa: a combinação entre a extração dos 21 pontos de referência da mão pelo MediaPipe e um classificador *Random Forest* mostrou-se adequada para reconhecer sinais estáticos da Libras em tempo real, sem depender de sensores ou equipamentos especializados.

5.1 Síntese dos resultados

O sistema atendeu integralmente a 11 dos 12 requisitos definidos e parcialmente ao restante (RNF04), com 100% de aprovação nos testes funcionais. O classificador alcançou 99% de acurácia no conjunto de teste, formado por 1.768 amostras (20% das 8.839 do *dataset*), e 98,57% na validação cruzada. Todos os números tiveram revocação de 1,00, as letras com movimento (H, J, K, X e Z) obtiveram *F1-score* entre 0,98 e 1,00, e os menores valores foram a revocação das letras R (0,94) e J (0,95) e a precisão do número 9 (0,94). O processamento de um quadro com a mão visível levou, em média, 73,9 ms no servidor, o que corresponde a cerca de 13 quadros por segundo e foi suficiente para o uso em tempo real.

Os erros pontuais observados no relatório de classificação não se traduziram em confusões perceptíveis durante o uso, em razão do limiar de confiança, da votação entre as últimas predições, da exigência de permanência estável do gesto e da separação entre os modos Letras e Números. Ao longo do projeto, a ampliação do *dataset* com mãos de diferentes pessoas mostrou-se decisiva: no início, a pouca variedade de mãos gerava erros no reconhecimento de algumas letras, e a gravação de mais tipos de mão reduziu esses erros. A integração com o VLibras, por sua vez, permitiu que o sistema cobrisse também o sentido inverso, de texto para Libras, sem a necessidade de desenvolver um tradutor próprio.

5.2 Limitações

O trabalho está restrito a sinais estáticos, de modo que não traduz palavras e frases em Libras, nem considera expressões faciais ou o uso simultâneo das duas mãos. A tradução de texto para Libras depende de conexão com a internet e de um serviço de terceiros, e o atraso de cerca de um segundo para confirmar cada caractere, adotado para reduzir erros, torna a digitação mais lenta. O *dataset* é desbalanceado, e a divisão aleatória das amostras pode ter deixado quadros muito parecidos no treino e no teste, o que torna a acurácia de 99% otimista em relação ao uso por pessoas que não participaram da coleta. O alcance de uso foi de cerca de 1 metro e variou conforme a câmera, já que algumas câmeras dificultam a leitura da mão. Por fim, o sistema foi avaliado por uma única pessoa externa ao grupo, sem a participação de pessoas surdas, e a comunicação entre o *front-end* e o *back-end* ocorre por HTTP, o que é adequado ao ambiente local de testes, mas deve ser substituído por HTTPS em uma implantação real.

5.3 Contribuições e trabalhos futuros

Como contribuição, o trabalho apresenta uma solução de baixo custo, que funciona com uma *webcam* comum, reúne em uma mesma interface o reconhecimento de letras e números e a tradução de texto para Libras, e documenta as decisões técnicas, os testes e as limitações do processo, o que pode servir de base para outros estudantes e pesquisadores. Em um contexto em que a Libras é reconhecida oficialmente, mas ainda pouco conhecida pela população em geral, a proposta busca contribuir para a redução da barreira de comunicação entre pessoas surdas e ouvintes.

Como trabalhos futuros, sugerem-se: avaliar o modelo com a separação das amostras por pessoa; ampliar e diversificar o *dataset*, com mais tipos de mão e câmeras diferentes; utilizar modelos sequenciais, que considerem vários quadros consecutivos, para reconhecer sinais dinâmicos, palavras e frases; incluir o uso das duas mãos; melhorar o reconhecimento a maiores distâncias; adotar HTTPS na comunicação; e realizar testes de usabilidade com um número maior de usuários, incluindo pessoas surdas e intérpretes de Libras, de modo a validar o sistema junto ao público ao qual ele se destina.

REFERÊNCIAS

Resumo das fontes consultadas

Quadro 7 – Fontes consultadas e onde foram utilizadas no trabalho

Nº	Fonte	Tipo	Assunto	Seções
1	Brasil (2002)	Lei	Reconhecimento legal da Libras (Lei nº 10.436)	1; 1.1; 2.1
2	Brasil (2005)	Decreto	Regulamentação da Lei da Libras (Decreto nº 5.626)	1; 2.1
3	Brasil (2018)	Lei	Lei Geral de Proteção de Dados Pessoais (LGPD)	2.8
4	Breiman (2001)	Artigo	Algoritmo Random Forest (floresta aleatória)	2.5
5	Gonzalez e Woods (2010)	Livro	Visão computacional e processamento de imagens	2.2
6	Haykin (2009)	Livro	Aprendizado de máquina e aprendizado supervisionado	2.4
7	IBGE (2010)	Censo	Pessoas com perda auditiva no Brasil	1; 1.1
8	OMS (2021)	Relatório	Perda auditiva no mundo	1; 1.1
9	Pedregosa et al. (2011)	Artigo	Biblioteca Scikit-learn	2.5
10	Pressman e Maxim (2016)	Livro	Engenharia de software e arquitetura de sistemas	2.7
11	Quadros e Karnopp (2004)	Livro	Estudos linguísticos da língua de sinais brasileira	2.1
12	Selvaraj et al. (2022)	Artigo	Reconhecimento de sinais a partir de pontos de pose (OpenHands)	1; 2.3; 2.6; 4.4
13	Sincan e Keles (2020)	Artigo	Base de dados e métodos para reconhecimento de língua de sinais (AUTSL)	1; 2.3; 2.6; 4.4
14	Sommerville (2011)	Livro	Engenharia de software e arquitetura cliente-servidor	2.7
15	Stallings (2015)	Livro	Criptografia e segurança de redes	2.8
16	Strobel (2008)	Livro	Cultura surda e história da comunidade surda	1.1; 1.2
17	Zhang et al. (2020)	Artigo	MediaPipe Hands: detecção da mão e 21 pontos de referência	1.1; 2.3

Fonte: Elaborado pelos autores (2026).

Links de acesso

1. Brasil (2002): https://www.planalto.gov.br/ccivil_03/leis/2002/l10436.htm

2. Brasil (2005): https://www.planalto.gov.br/ccivil_03/_ato2004-2006/2005/decreto/d5626.htm

3. Brasil (2018): https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm

4. Breiman (2001): https://doi.org/10.1023/A:1010933404324

7. IBGE (2010): https://censo2010.ibge.gov.br/

8. OMS (2021): https://www.who.int/publications/i/item/world-report-on-hearing

9. Pedregosa et al. (2011): https://jmlr.org/papers/v12/pedregosa11a.html

12. Selvaraj et al. (2022): https://aclanthology.org/2022.acl-long.150

13. Sincan e Keles (2020): https://doi.org/10.1109/ACCESS.2020.3028072

17. Zhang et al. (2020): https://arxiv.org/abs/2006.10214

Obs.: as fontes sem link (nºs 5, 6, 10, 11, 14, 15 e 16) são livros impressos.

Lista de referências

BRASIL. **Decreto nº 5.626, de 22 de dezembro de 2005**. Regulamenta a Lei nº 10.436, de 24 de abril de 2002, que dispõe sobre a Língua Brasileira de Sinais – Libras, e o art. 18 da Lei nº 10.098, de 19 de dezembro de 2000. Brasília, DF: Presidência da República, 2005. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2004-2006/2005/decreto/d5626.htm. Acesso em: 29 out. 2026.

BRASIL. **Lei nº 10.436, de 24 de abril de 2002**. Dispõe sobre a Língua Brasileira de Sinais – Libras e dá outras providências. Brasília, DF: Presidência da República, 2002. Disponível em: https://www.planalto.gov.br/ccivil_03/leis/2002/l10436.htm. Acesso em: 29 out. 2026.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 29 out. 2026.

BREIMAN, Leo. Random forests. **Machine Learning**, v. 45, n. 1, p. 5–32, 2001. DOI: 10.1023/A:1010933404324. Disponível em: https://doi.org/10.1023/A:1010933404324. Acesso em: 29 out. 2026.

GONZALEZ, Rafael C.; WOODS, Richard E. **Processamento digital de imagens**. 3. ed. São Paulo: Pearson Prentice Hall, 2010.

HAYKIN, Simon. **Neural networks and learning machines**. 3. ed. Upper Saddle River: Pearson, 2009.

INSTITUTO BRASILEIRO DE GEOGRAFIA E ESTATÍSTICA (IBGE). **Censo Demográfico 2010**. Rio de Janeiro: IBGE, 2010. Disponível em: https://censo2010.ibge.gov.br/. Acesso em: 29 out. 2026.

ORGANIZAÇÃO MUNDIAL DA SAÚDE (OMS). **World report on hearing**. Genebra: OMS, 2021. Disponível em: https://www.who.int/publications/i/item/world-report-on-hearing. Acesso em: 29 out. 2026.

PEDREGOSA, Fabian et al. Scikit-learn: machine learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825–2830, 2011. Disponível em: https://jmlr.org/papers/v12/pedregosa11a.html. Acesso em: 29 out. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

QUADROS, Ronice Müller de; KARNOPP, Lodenir Becker. **Língua de sinais brasileira**: estudos linguísticos. Porto Alegre: Artmed, 2004.

SELVARAJ, Prem et al. OpenHands: making sign language recognition accessible with pose-based pretrained models across languages. *In*: ANNUAL MEETING OF THE ASSOCIATION FOR COMPUTATIONAL LINGUISTICS, 60., 2022, Dublin. **Proceedings** [...]. Dublin: ACL, 2022. p. 2114–2133. DOI: 10.18653/v1/2022.acl-long.150. Disponível em: https://aclanthology.org/2022.acl-long.150. Acesso em: 29 out. 2026.

SINCAN, Ozge Mercanoglu; KELES, Hacer Yalim. AUTSL: a large scale multi-modal Turkish Sign Language dataset and baseline methods. **IEEE Access**, v. 8, p. 181340–181355, 2020. DOI: 10.1109/ACCESS.2020.3028072. Disponível em: https://doi.org/10.1109/ACCESS.2020.3028072. Acesso em: 29 out. 2026.

SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

STALLINGS, William. **Criptografia e segurança de redes**: princípios e práticas. 6. ed. São Paulo: Pearson, 2015.

STROBEL, Karin. **As imagens do outro sobre a cultura surda**. Florianópolis: Ed. da UFSC, 2008.

ZHANG, Fan et al. **MediaPipe Hands**: on-device real-time hand tracking. arXiv:2006.10214, 2020. Disponível em: https://arxiv.org/abs/2006.10214. Acesso em: 29 out. 2026.